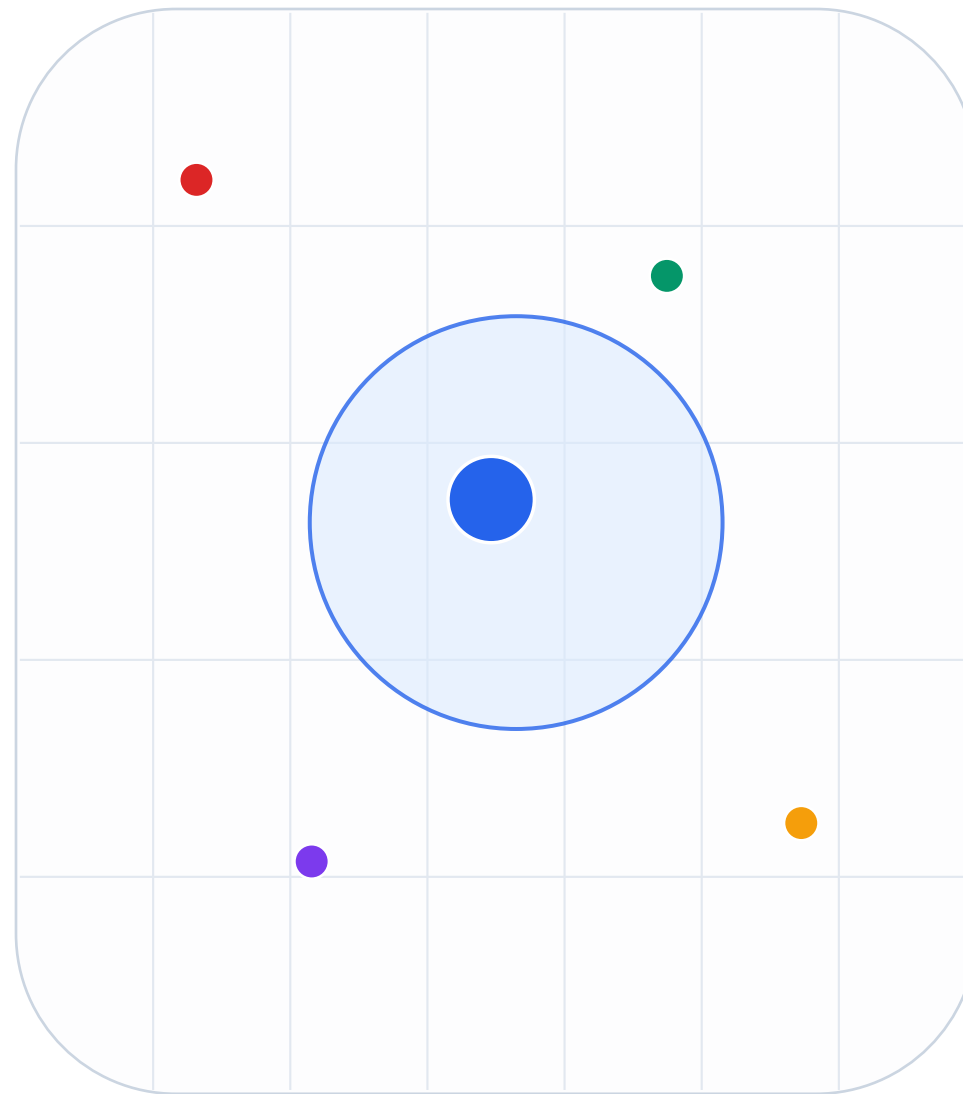


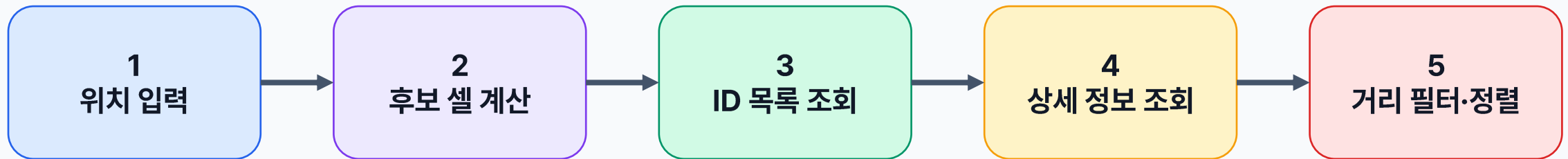
CHAPTER 01

근접성 서비스

현재 위치 근처의 음식점·호텔·주유소 같은 POI를 빠르게 찾는 시스템 설계



오늘의 설계 질문



문제 이해 및 범위 정의

사용자 검색 경로

- 사용자 위치와 검색 반경에 맞는 사업장 목록 반환
- 카테고리과 같은 기본 검색 옵션 지원
- 사업장 상세 정보 조회 가능

사업장 관리 경로

- 사업장 소유자가 정보 추가 / 삭제 / 수정
- 검색 결과에 즉시 반영될 필요는 낮음
- DB의 정합성을 우선하고 색인은 비동기 반영 가능

이번 장에서 제외

- 길찾기 / ETA / 지도 타일 렌더링
- 추천 랭킹, 리뷰 피드, 광고 경매
- 주변 친구처럼 실시간 위치 스트리밍

비기능 요구사항



낮은 지연

검색 요청은 사용자의
즉시 행동과 연결된다.
셀 후보 조회 / 캐시 / 정렬 경로를
짧게 만든다.



개인정보 보호

위치 정보는 민감 데이터다.
동의, 최소 보관, 접근 통제,
지역별 규정을 고려한다.



고가용성 / 확장성

도심 / 점심시간 / 이벤트 지역에서
트래픽이 몰린다.
무상태 서비스와 복제본으로
읽기를 확장한다.

대략적 규모 추정

100M

DAU 가정

5회

사용자당 일 검색

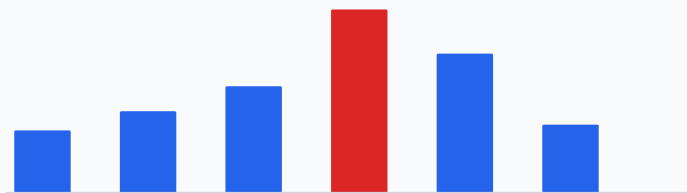
200M

등록 사업장

$$\text{QPS} \approx (100\text{M} \times 5) / 100\text{K sec} = 5,000$$

- 평균 QPS는 5K지만, 실제 설계에서는 피크 계수를 별도로 둔다.
- 검색 / 상세 조회가 대부분이므로 읽기 중심 시스템이다.
- 쓰기 요청은 사업장 CRUD에 집중되며 검색 색인 반영은 비동기로 처리할 수 있다.

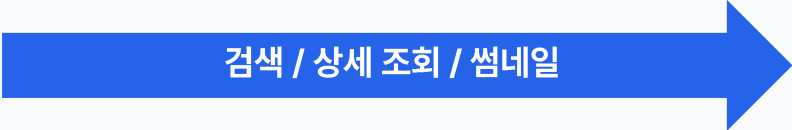
평균과 피크의 차이



점심 / 퇴근 / 이벤트 지역은
평균보다 훨씬 높은 QPS를 만든다.

설계 방향 : 읽기 최적화 + 위치 색인

READ



검색 / 상세 조회 / 썸네일

WRITE



사업장 CRUD

- 검색 서비스는 무상태로 두어 수평 확장한다.
- 사업장 쓰기는 primary DB로 모으고
읽기는 secondary DB / 캐시에서 처리한다.
- 검색 색인은 DB 테이블과 별도 캐시 구조로 만들 수 있다.

- 위 / 경도는 연속값이라 그대로 캐시 키로 쓰기 어렵다.
- 지오해시 / 쿼드트리 같은 셀 키가 반복 조회를 가능하게 한다.
- 후보 조회 후 실제 거리 계산으로 정확도를 보정한다.

API 설계

GET /v1/search/nearby

?lat=37.56&lng=126.98
&radius=500m&category=restaurant
&limit=20&cursor=...

GET /v1/businesses/{id}

사업장 상세 정보

주소, 영업시간, 사진, 평점

POST / PUT / DELETE

사업장 소유자의 정보 관리

검색 색인은 비동기 반영 가능

응답 예시

```
{
  "items": [
    { "business_id": "b_1001",
      "name": "을지로 파스타",
      "distance_m": 182,
      "rating": 4.6,
      "category": "restaurant",
      "lat": 37.566, "lng": 126.982 },
    ...
  ],
  "next_cursor": "eyJwYWdl..."
}
```

데이터 모델

테이블	주요 필드	역할
business	business_id, name, address, lat, lng, category, owner_id	사업장 상세 정보의 원본
geospatial_index	geo_key, business_id, lat, lng	셀 키 → 후보 ID 목록 조회
business_hours	business_id, day, open, close	영업 상태 필터
category_index	category, business_id	카테고리 필터 보조

business
원본 데이터

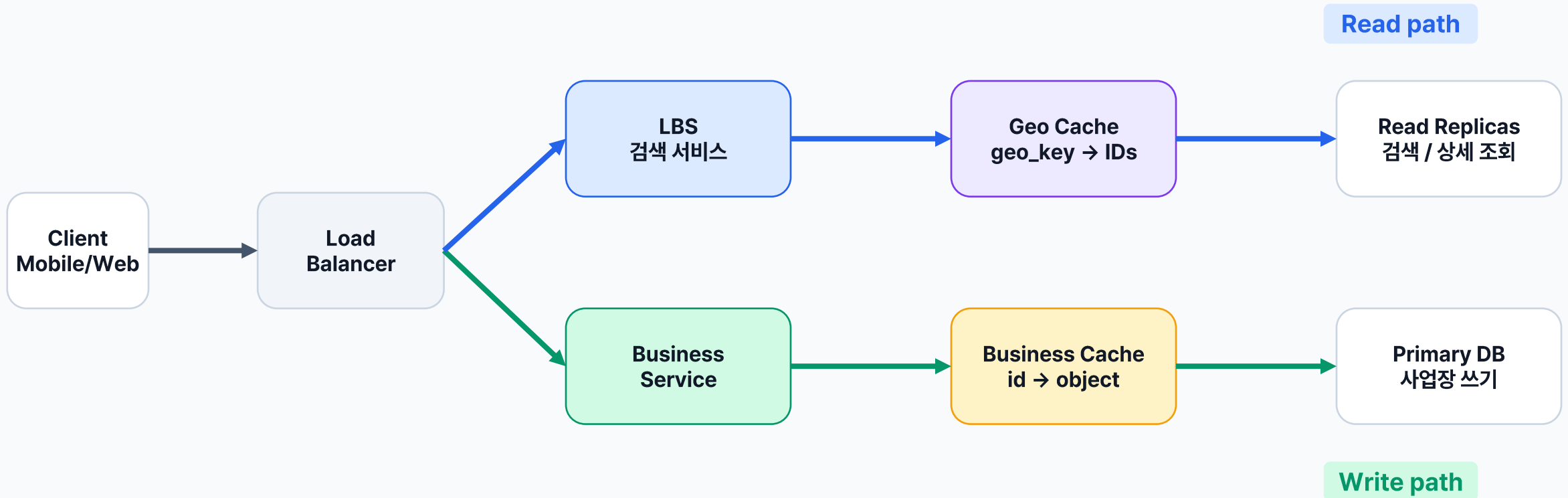
geospatial_index
셀 키

business_id
객체 캐시

geo_key
ID 목록 캐시

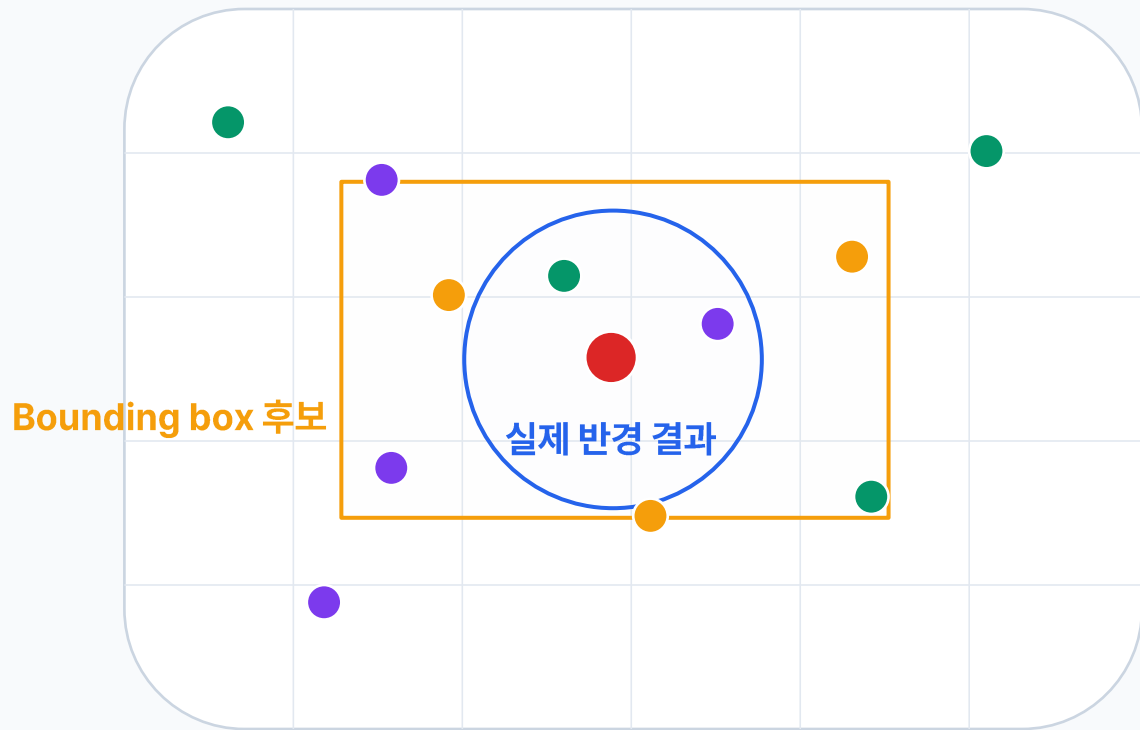
검색 경로는 먼저 geo_key로 후보 ID를 찾고,
그 다음 business_id로 상세 객체를 가져온다.

개략적 아키텍처



- LBS는 무상태로 수평 확장한다.
- 캐시는 geo_key 기반 후보 목록과 business_id 기반 상세 객체로 나눈다.
- 사업장 쓰기는 primary DB, 읽기는 replica / 캐시를 활용한다.

왜 단순 위 / 경도 DB 검색은 부족한가



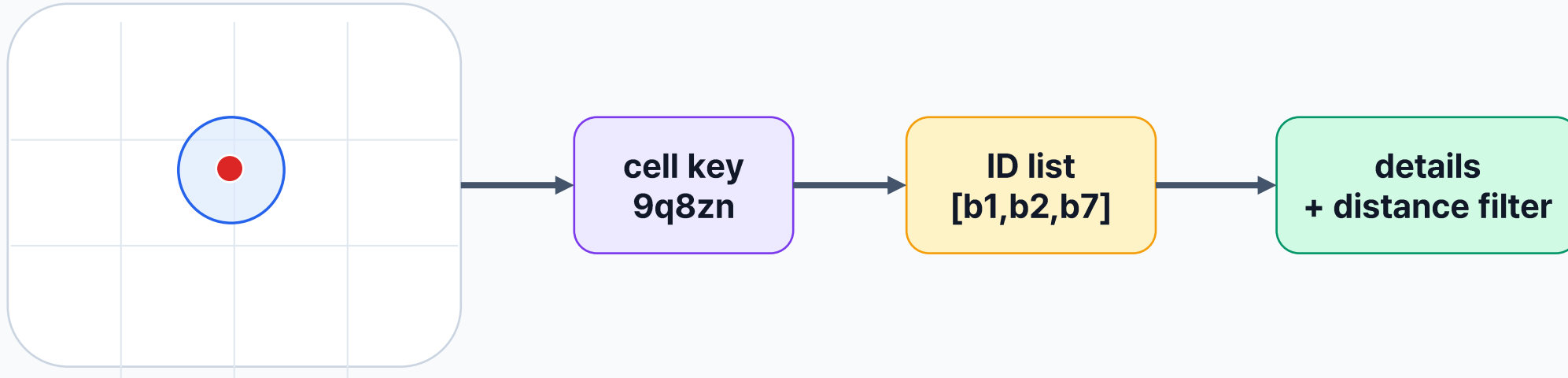
문제

- SQL로 lat BETWEEN ... AND lng BETWEEN ... 를 걸면 우선 사각형 후보가 나온다.
- 후보가 많은 도심 지역에서는 거리 계산 대상이 급격히 늘어난다.
- 위도와 경도 각각의 인덱스만으로 "원 안의 점" 검색을 충분히 줄이기 어렵다.

해결 방향

- 지도를 셀로 나누고, 좌표를 셀 키로 변환한다.
- 셀 키로 후보 ID 목록을 가져온 뒤, 실제 거리 계산으로 정확도를 보정한다.

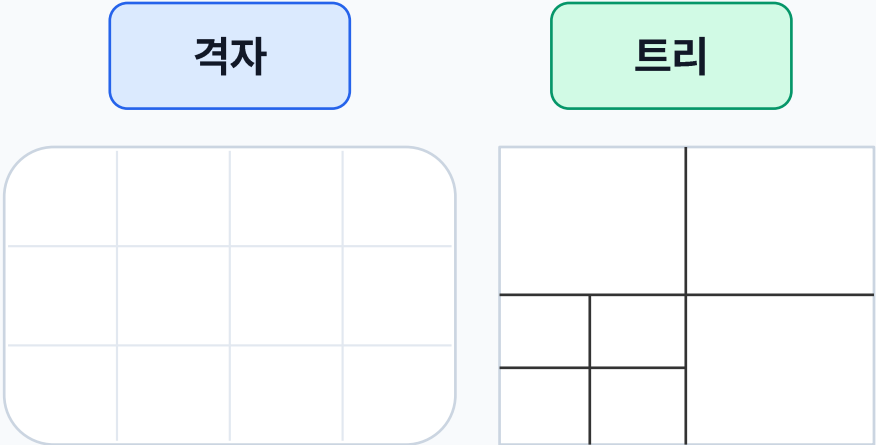
위치 색인의 공통 아이디어



- 셀 키는 많은 유사 위치 요청을 같은 캐시 키로 묶어준다.
- 셀 후보에는 반경 밖의 사업장도 포함될 수 있으므로 마지막 거리 필터가 필요하다.
- 검색 반경과 셀 크기가 맞지 않을 때는 인접 셀도 함께 조회한다.

색인 후보 비교

방식	강점	주의점
균등 격자	구현 가장 단순	데이터 분포 불균등 / 인접 셀 체계 약함
지오해시	문자열 접두사와 캐시 친화성	경계 문제 / 정밀도 선택 필요
쿼드트리	밀집 지역을 더 잘게 나눔	메모리 상주 / 재구축 / 배포 고려
S2 / R-tree	지구 곡률 / 다각형 / geofence에 강함	구현 복잡도와 라이브러리 의존



지오해시 동작 방식



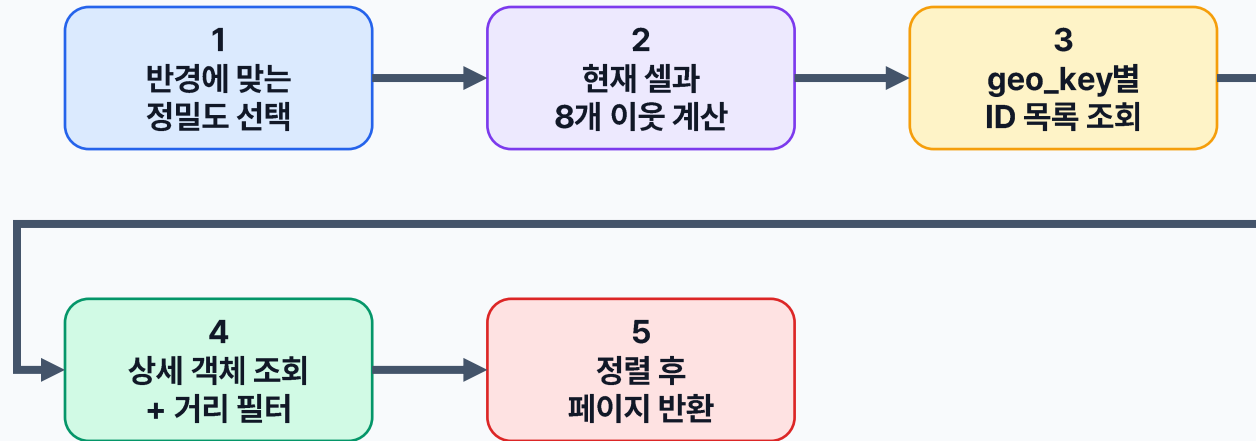
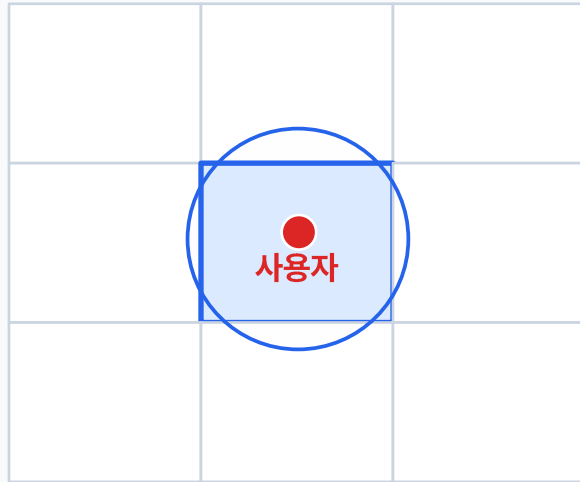
u4pru

u4pru
u4pruy
u4pruyd

문자가 길수록 작은 셀

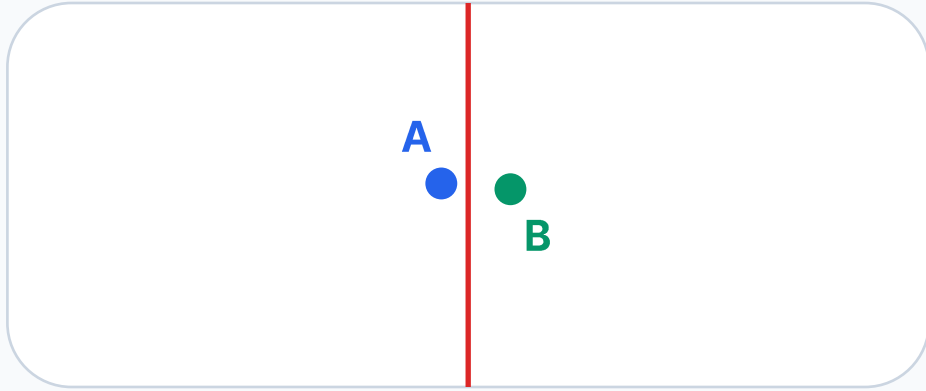
- 각 접두사는 지도상의 직사각형 셀을 의미한다.
- 같은 접두사를 공유하면 보통 서로 가까운 후보로 볼 수 있다.
- 하지만 “가깝다 ⇒ 같은 접두사”는 항상 성립하지 않으므로 경계 처리가 필요하다.

지오해시 검색 흐름



- 정밀도가 너무 낮으면 후보가 많고, 너무 높으면 여러 셀을 많이 조회해야 한다.
- 반드시 실제 거리 계산으로 원 밖의 후보를 제거한다.

지오해시의 두 가지 함정



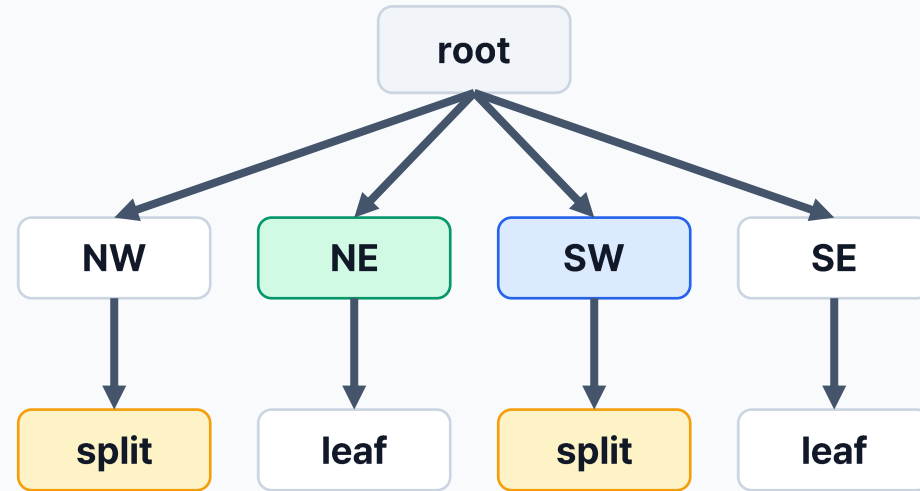
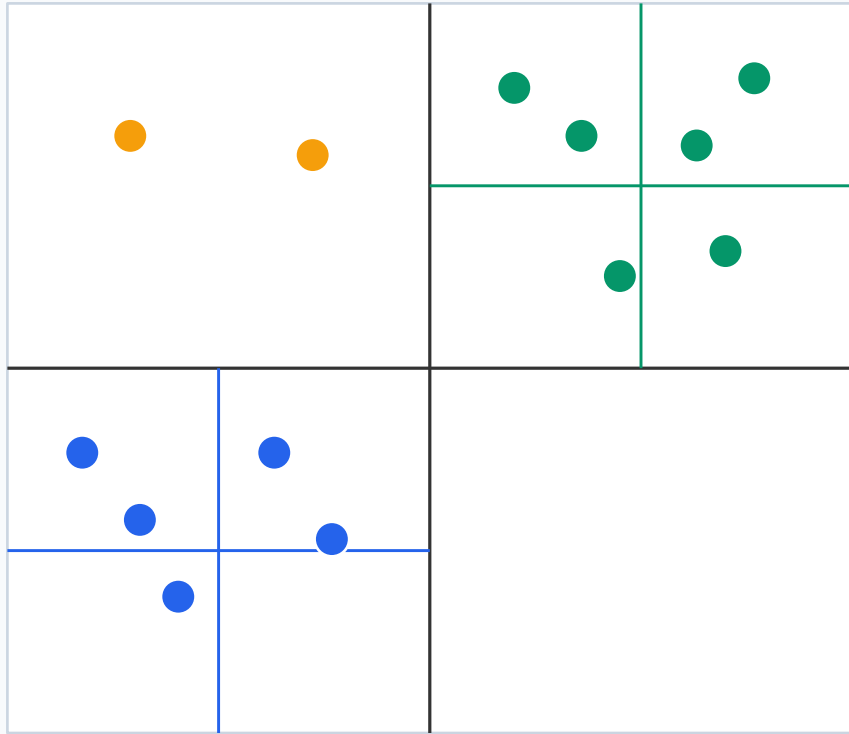
가깝지만 셀 경계가 다름



마지막 문자를 줄이면 더 큰 셀

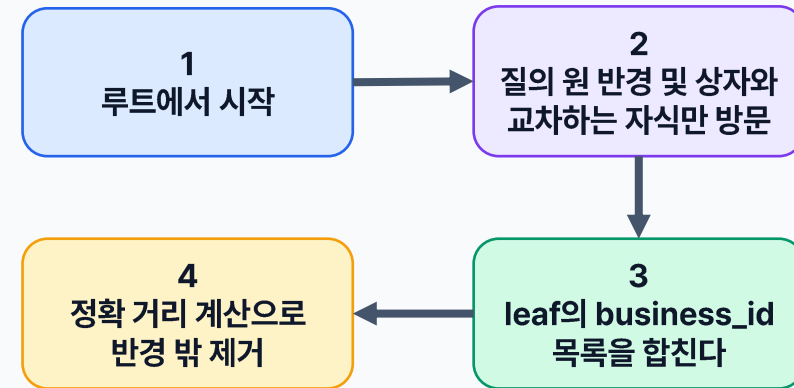
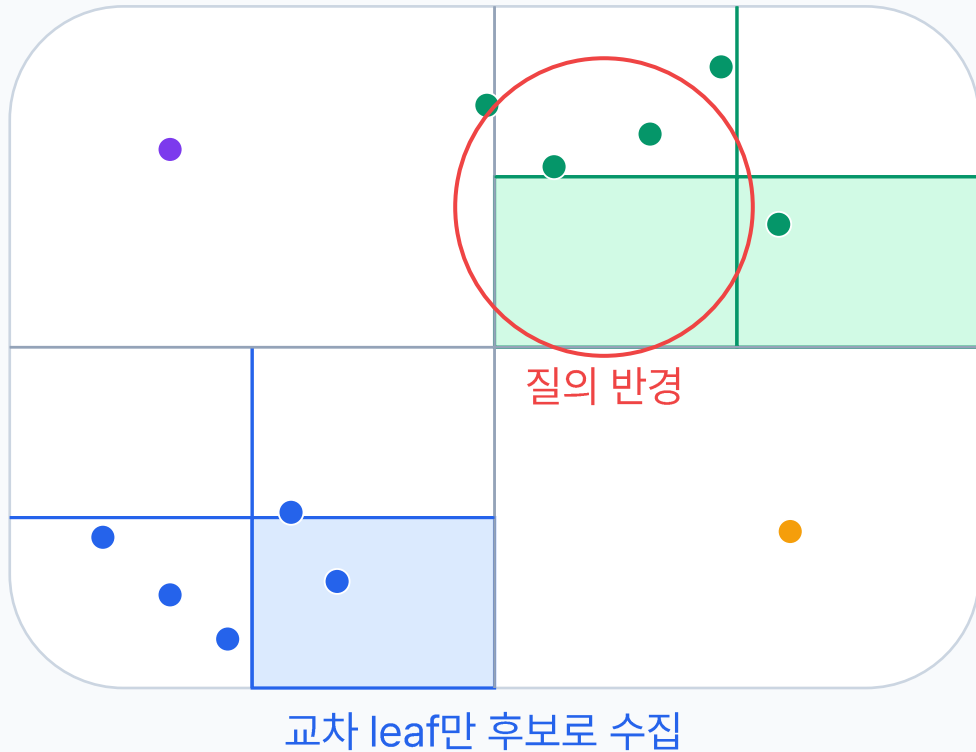
- 경계 문제 : 현재 셀만 조회하면 실제로 가까운 사업장을 놓칠 수 있다 → 인접 셀까지 조회한다.
- 결과 부족 : 반경 안 후보가 너무 적으면 정밀도를 낮추거나 반경을 넓혀 후보를 더 가져온다.
- 후보 확장은 정확도보다 재호출을 우선하고, 최종 정확도는 거리 계산으로 보정한다.

쿼드트리 동작 방식



- 각 노드는 사분면을 나타내고, leaf가 임계치 이하의 사업장을 담을 때까지 나눈다.
- 도심처럼 밀집한 영역은 깊어지고, 바다 / 산처럼 밀집하지 않은 영역은 얕다.
- 검색은 사용자 위치가 포함된 leaf부터 시작해 필요한 이웃 노드를 확장한다.

쿼드트리 심화 : 교차 Leaf만 방문



- Threshold를 넘는 leaf만 4분할한다.
- 도심은 깊은 트리, 교외는 얇은 트리가 된다.
- 범위 검색은 '교차하는 leaf'만 따라가므로 후보 수가 줄어든다.
- KNN은 가까운 leaf부터 확장하는 방식으로 설명할 수 있다.
- 업데이트가 많으면 split/merge, 재구축, 버전 전환 비용이 든다.

쿼드트리 운영 고려사항

≤100

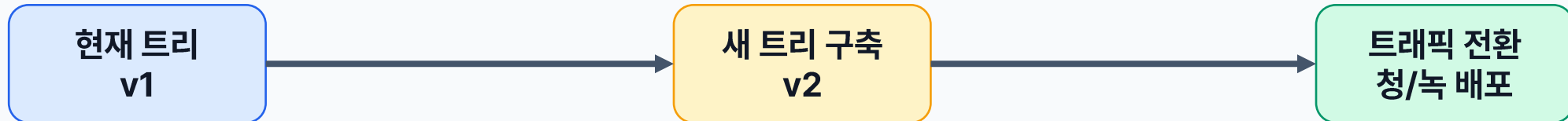
leaf 당 사업장 기준 예시

~2M

leaf node 수 예시

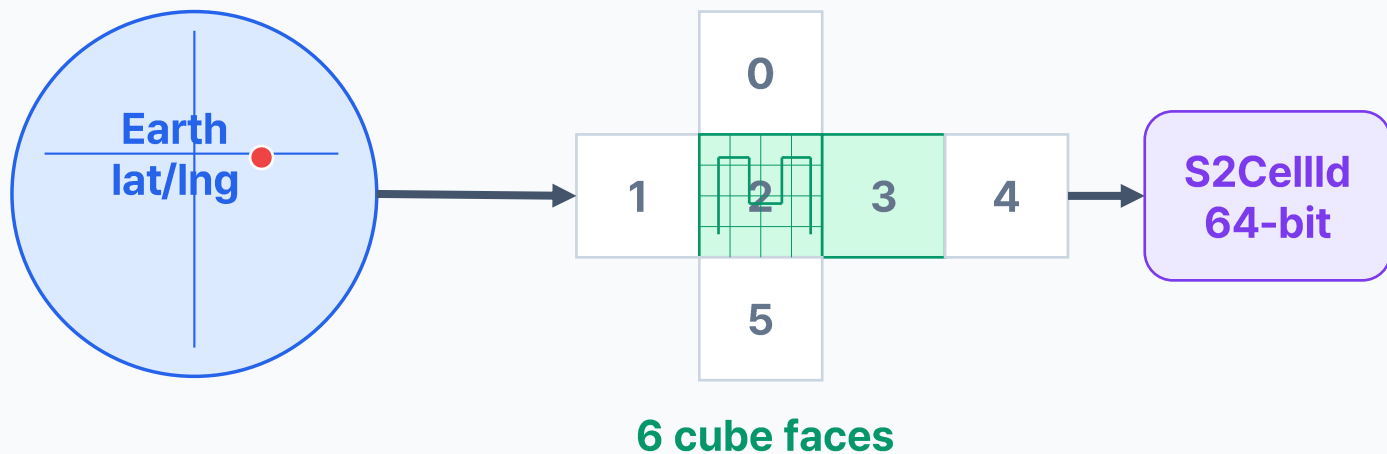
~1.7GB

책의 메모리 추정 예시



- 쿼드트리를 메모리에 올리면 조회는 빠르지만, 서버 시작 / 재구축 시간이 길어질 수 있다.
- 사업장 변경이 잦지 않다면 정기 배치 재구축이 단순하다.
- 무중단 전환을 위해 새 트리를 만든 뒤 트래픽을 옮기는 배포 전략이 필요하다.

S2 Geometry : 지구를 계층적 Cell ID로 표현



동작 방식

1. Cell level
level이 높을수록 더 작은 셀 / 반경에 맞춰 level 선택

2. Hilbert order
가까운 셀이 비슷한 ID 범위에 오도록 locality를 보존

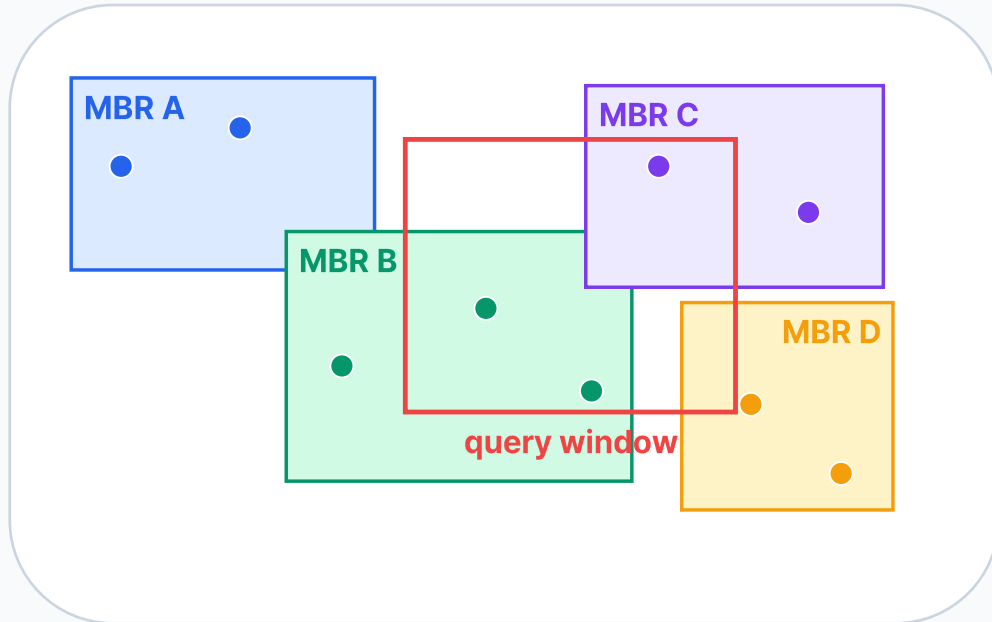
3. Region cover
원의 영역을 여러 S2 cell로 덮어 후보 key를 만든다

4. Refine
cell 후보에는 오차가 있으므로 거리 계산으로 보정

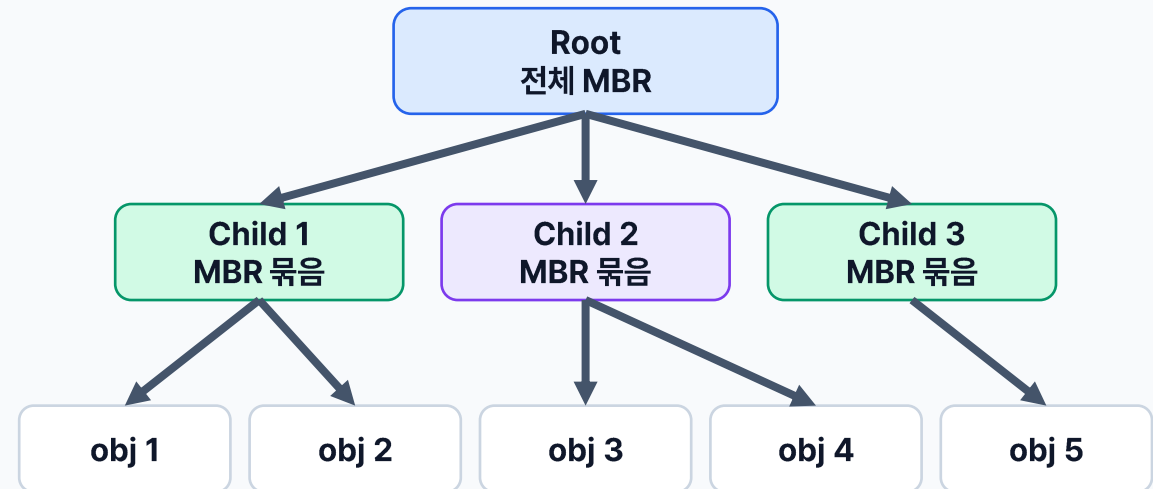
검색 흐름 예시



R-tree : Bounding Box 기반 DB 인덱스



- Index hit : B, C, D
- Exact hit : 실제 geometry 계산



R-tree 검색의 2단계

1. Index Filter : query window와 겹치는 최소 경계 사각형(MBR)만 탐색
2. Refine : 후보 객체에 ST_DWithin / ST_Intersects 같은 정확 계산 적용
 1. ST_Dwithin : 두 공간 객체 간의 거리가 지정한 임계값 이내인지 판별
 2. ST_Intersects : 두 공간 객체가 교차하는지 판별

색인 선택 기준 : 단순성 vs 표현력

Geohash

- 문자열 접두사로 DB / 캐시 키화가 쉽다.
- Redis, RDBMS, 검색엔진과 연동이 비교적 단순하다.
- 경계 / 정밀도 문제를 애플리케이션에서 처리해야 한다.

Quadtree

- 밀집도에 맞춰 셀 크기가 달라진다.
- k-nearest 또는 “충분한 후보 수” 확보에 유리하다.
- 메모리 상주, 재구축, 무종단 배포가 설계 비용이다.

캐시/구현 단순성

밀도 적응 / KNN

캐시 전략 : 위치는 캐시 키가 되기 어렵다

나쁜 키

37.566321,126.982701
37.566322,126.982699
37.566410,126.982850

좋은 키

geo:9q8zn
→ [b1, b9, b77, ...]

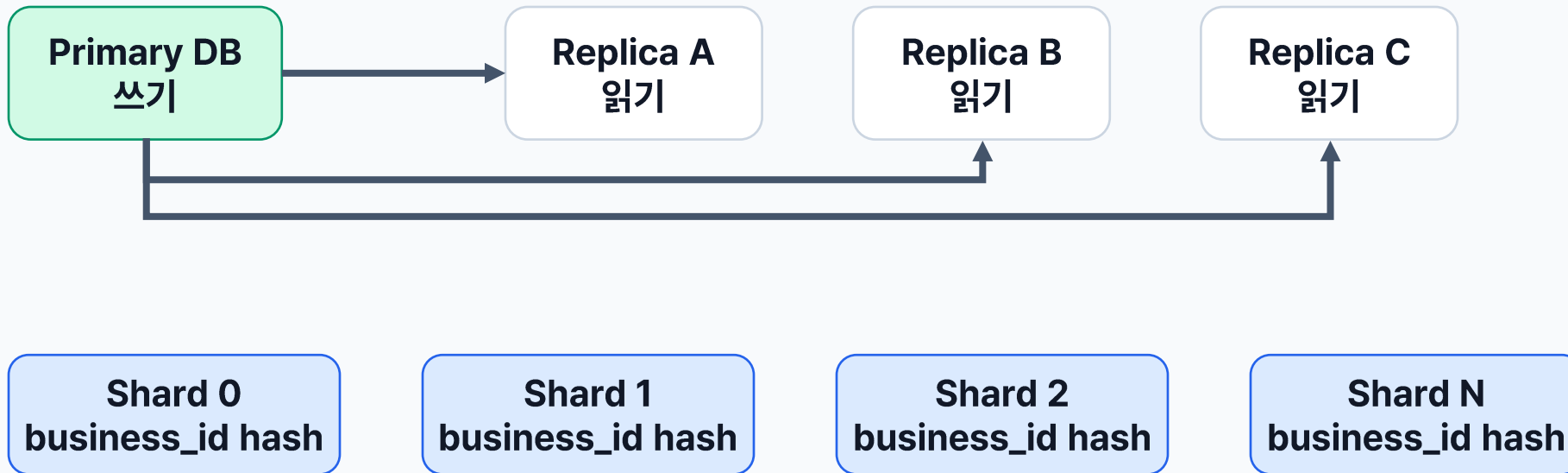
biz:b1
→ {name, photo, rating}

Cache 1
셀 → 사업장 ID 목록

Cache 2
사업장 ID → 상세 객체

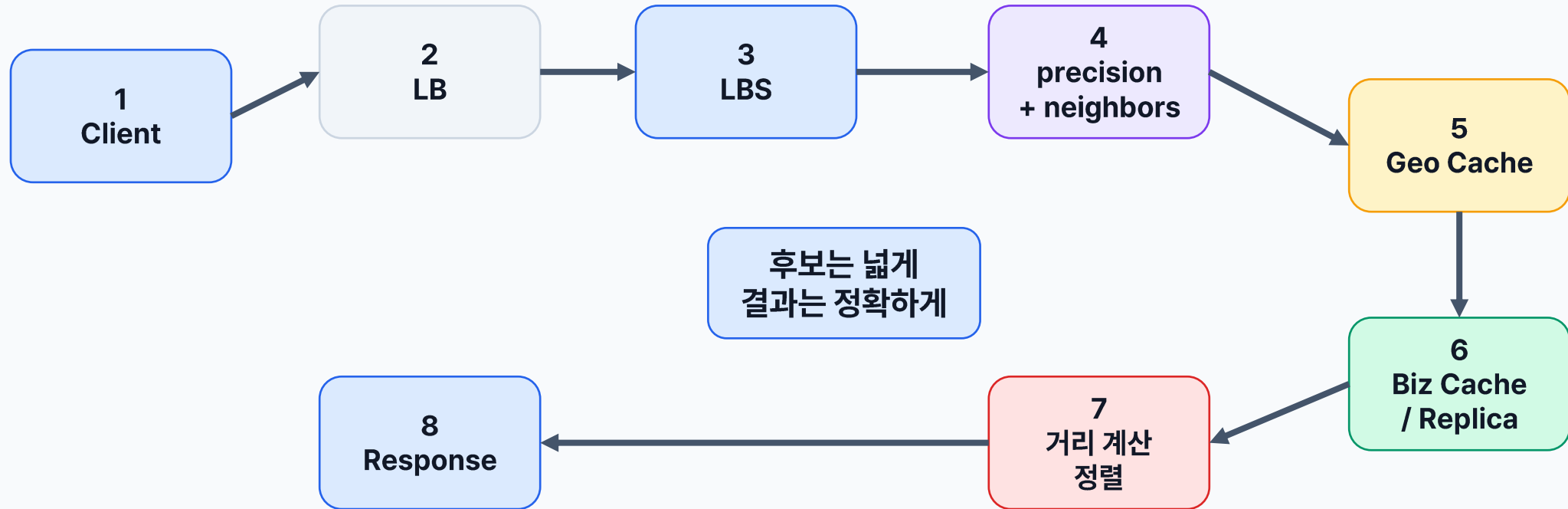
- 셀 단위 ID 목록은 비교적 작고 반복 조회가 많아 캐시 적중률이 좋다.
- 상세 객체 캐시는 검색뿐 아니라 상세 조회 API에서도 재사용된다.
- 업데이트 시 TTL, version key, write-through / async invalidation 중 하나를 선택한다.

DB 확장 : 샤딩과 복제



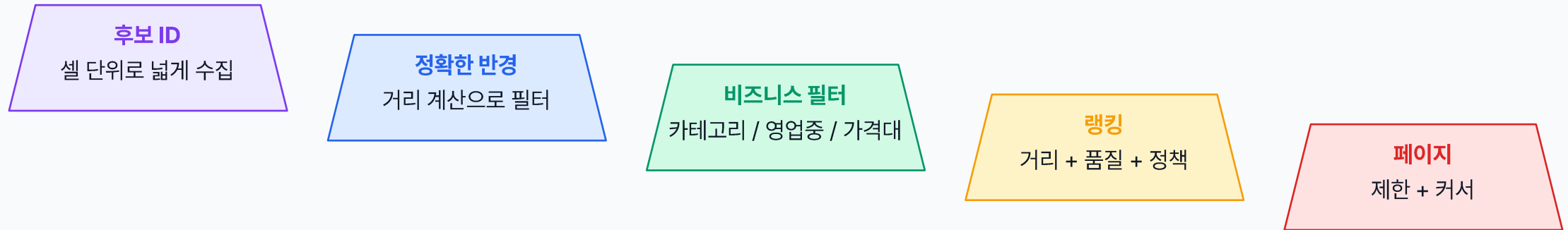
- business 테이블은 business_id 기준 해시 샤딩이 단순하다.
- geo_key 기준 샤딩은 핫 셀, 인접 셀 조회, 리샤딩 문제가 있어 더 조심스럽다.
- 검색 경로의 병목은 DB가 아니라 캐시 / 복제 / 인덱스 설계로 흡수한다.

주변 검색 상세 시나리오



- LBS는 geo_key 목록을 만든 뒤 캐시 미스일 때 replica를 조회한다.
- business_id 목록은 중복 제거 후 상세 객체로 확장된다.
- 반경 / 카테고리 / 영업 상태를 필터링하고 안정적인 정렬 키로 커서를 만든다.

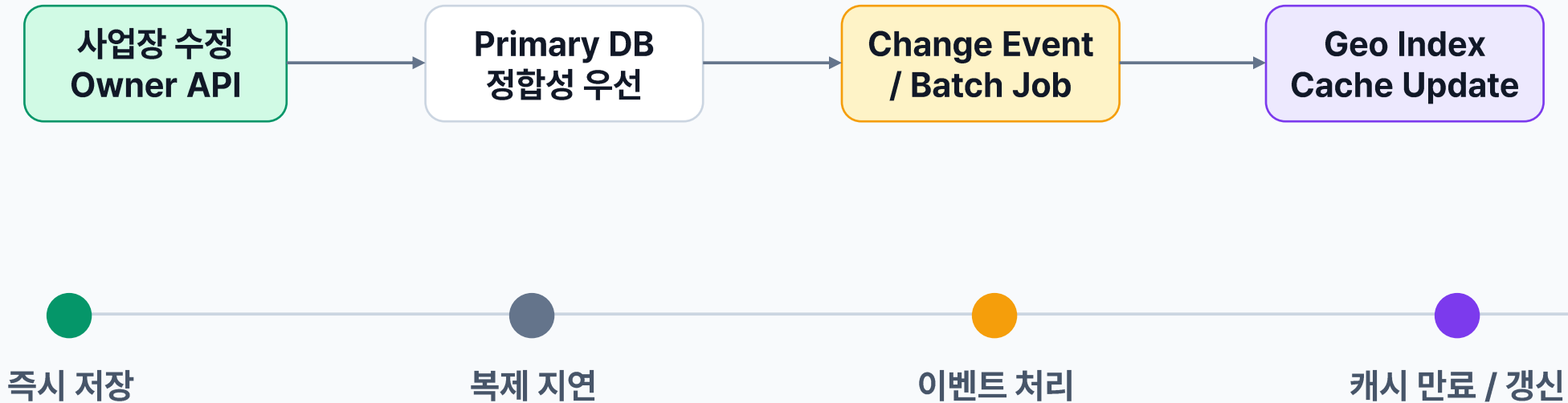
정렬 / 페이지네이션 : 검색 결과를 제품처럼 만들기



주의할 점

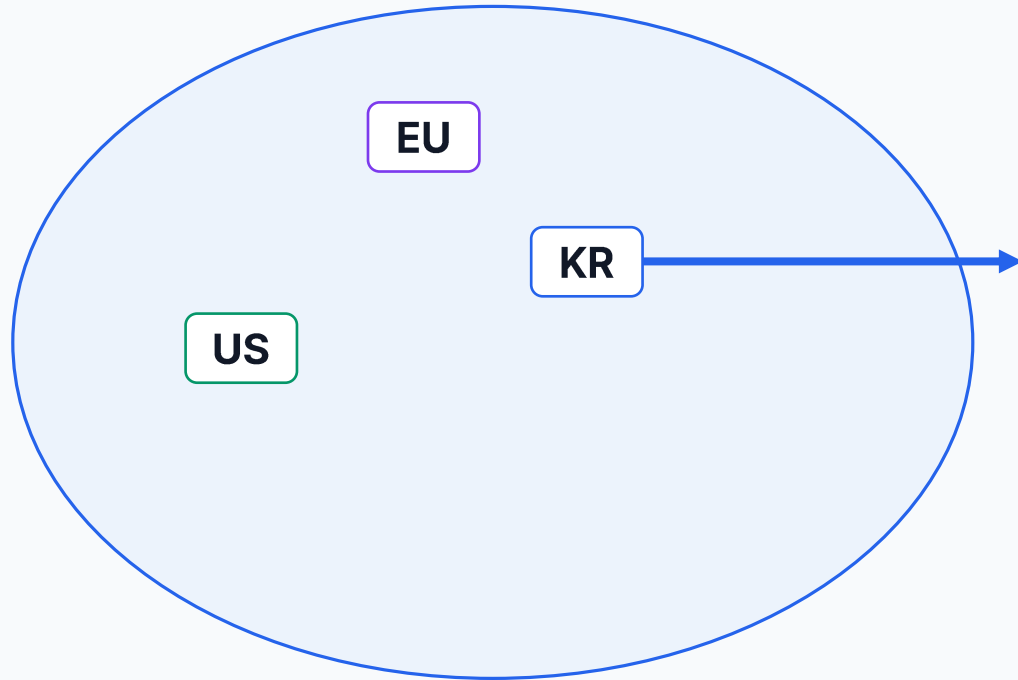
- 랭킹을 검색 색인에 과도하게 넣으면 업데이트와 캐시 무효화가 복잡해진다.
- 거리순만 쓰면 품질이 낮은 결과가 상단에 올 수 있고, 평점순만 쓰면 멀리 있는 결과가 올라올 수 있다.
- 커서는 "1차 정렬 기준 + 2차 정렬 기준"를 포함해야 페이지가 흔들리지 않는다.

일관성 : 실시간 반영이 아니어도 되는 이유



- 원본 DB의 정합성은 강하게, 검색 색인의 최신성은 요구사항에 맞춰 완화한다.
- 검색 결과가 몇 분 늦게 반영되어도 괜찮다면 비동기 갱신이 비용을 크게 낮춘다.
- 삭제 / 폐업 / 위험 정보처럼 민감한 변경은 일반 업데이트보다 빠른 무효화 경로를 둘 수 있다.

멀티 리전과 개인정보



Regional LBS
+ Cache

Regional Replica
/ Data Boundary

설계 포인트

- GeoDNS / 라우팅으로 가까운 리전에 보낸다.
- 리전별 캐시와 읽기 복제본으로 지연을 줄인다.
- 지역별 데이터 거주성 / 개인정보 규정을 반영한다.
- 정확한 실시간 위치는 필요할 때만 처리하고 장기 보관하지 않는다.

장애 대응과 관측성

Cache miss storm

핫 셀 TTL 만료가 동시에 발생

Replica lag

사업장 정보가 오래된 상태로 조회

Hot geohash

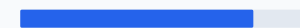
도심 / 이벤트 지역에 요청 집중

Bad index build

새 색인 배포 후 결과 누락

핵심 지표

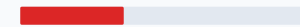
p95 / p99 latency



cache hit ratio



empty result rate



hot cell QPS



index update delay



- 캐시 TTL jitter와 single-flight로 동시에 원본 DB로 몰리는 현상을 줄인다.
- 새 색인은 shadow read / canary로 검증한 뒤 전환한다.
- 결과가 비정상적으로 적은 셀, 특정 리전 지연, hot cell QPS를 따로 모니터링한다.

면접에서 강조할 트레이드오프

Geohash		Quadtree / S2	단순성 vs 밀도 적응
낮은 지연		최신 데이터	캐시 활용 vs stale 가능성
넓은 후보		적은 후보	재호출 vs 비용
ID 샤딩		Geo 샤딩	쓰기 단순성 vs 지역성
개인화		개인정보 보호	정확한 위치 활용 vs 최소 수집

스터디 토론 질문

Q1

한국처럼 도심 밀도가 높고 지역 편차가 큰 서비스라면, 지오해시와 쿼드트리 중 무엇을 1차 선택으로 둘 것인가? 선택 기준은 데이터 밀도, 업데이트 빈도, KNN 요구, 운영 난이도 중 무엇인가?

Q2

사업장 정보 수정이 “1분 이내 검색 결과 반영”을 요구한다면, 캐시 무효화와 지리공간 색인 갱신 구조를 어떻게 바꿀 것인가?

Q3

정확한 위치 기반 랭킹을 제공하면서도 개인정보 위험을 낮추려면, 로그 / 캐시 / 분석 파이프라인에서 어떤 데이터를 남기지 말아야 할까?

핵심 요약

1

2D 좌표를 셀 키로

위경도를 그대로 찾지 말고
geohash/quadtree 같은 색인으로 후보를 줄인다.

3

읽기 경로 최적화

검색은 캐시와 read replica 중심,
쓰기는 primary DB 중심으로 분리한다.

5

색인 선택은 트레이드오프

Geohash는 단순하고, Quadtree는 밀도 적응에 강하지만
운영 비용이 있다.

2

후보 조회 후 정확 필터

셀 후보에는 오차가 있으므로
실제 거리 계산으로 최종 반경을 보정한다.

4

캐시는 raw 좌표가 아닌 셀 키

geo_key → business_id 목록, business_id → 상세 객체
2단계가 유용하다.

6

운영·프라이버시 포함

핫 셀, 캐시 미스, 색인 지연, 위치 데이터 보호를 설계에 포함한다.

한 문장으로: 셀 기반 후보 검색 + 캐시/복제 읽기 경로 + 정확 필터링이 근접성 서비스의 뼈대다.